

Excepții în C# — pe codul tău

Teorie, exerciții ghidate, practică și provocări — pe academy.

Lecția asta nu pleacă de la teorie, ci de la o singură cifră din `academy` :

49 de `throw` în tot proiectul. Toate sunt `ArgumentException` . Trei `catch` -uri:
`Program.cs:12` , `ViewStudent.cs:192` , `ViewStudent.cs:218` . Toate pe același tip.

Nu e o scăpare de stil. E motivul pentru care, azi, nimeni din aplicația ta nu poate reacționa diferit la două lucruri diferite.

1. Tipul excepției e mesajul adevărat. Textul e doar o notă.

Sub singurul tău tip stau cinci situații care n-au nimic în comun:

Situația	Un exemplu din codul tău
validare de câmp	<code>Users/Models/User.cs:96</code> — „Emailul nu poate fi gol”
conflict / duplicat	<code>Books/Services/BookService.cs:33</code> — „Cartea se afla deja in baza de date”
ceva nu există	<code>Courses/Services/CourseService.cs:63</code> — „Cursul nu exista in baza de date”
tip necunoscut în fișier	<code>Users/Mappers/UserTextMapper.cs:74</code> — „Unknown user type”
credențiale greșite	<code>ViewLogIn.cs:50</code> — „Parola gresita!”

Când scrii `catch (ArgumentException)`, le prinzi pe toate cinci. Ca să afli care dintre ele s-a întâmplat, ți-ar rămâne o singură unealtă: să citești **textul** mesajului și să-l compari cu un șir. Adică să iei decizii de program pe baza unei propoziții în română.

Regula: situații care cer reacții diferite au nevoie de tipuri diferite. `catch` selectează după **tip**, nu după text — de asta tipul e informația, iar mesajul e pentru om.

2. Ce vede azi utilizatorul tău

Am pornit aplicația și am greșit parola. Asta a apărut pe ecran:

```
Parola: System.ArgumentException: Parola gresita!  
    at stefan_academy_vanilla_charp.ViewLogIn.Logger() in  
      C:\...\ViewLogIn.cs:line 50  
    at Program.Main() in C:\...\Program.cs:line 10
```

Un om care a greșit o literă primește numele clasei tale, calea absolută de pe disc și numărul liniei.

De ce se întâmplă: `Program.cs:14` scrie

```
catch (ArgumentException text)  
{  
    Console.WriteLine(text);  
}
```

`text` e **obiectul** excepție, nu mesajul lui. `Console.WriteLine` cheamă pe el `ToString()`, iar `Exception.ToString()` întoarce tip + mesaj + **stack trace**. Dacă scriai `text.Message`, ieșea doar „Parola gresita!”.

Aceeași greșeală, aceeași formă, în `ViewStudent.cs:192` și `:218` — `Console.WriteLine(ex)`.

Regula: `ex.Message` pentru utilizator, obiectul întreg doar într-un jurnal pe care îl citești tu.

3. Ierarhia: de ce o clasă de bază proprie

```
public abstract class AcademyException : Exception
public class ValidationException : AcademyException
public class NotFoundException : AcademyException
public class DuplicateException : AcademyException
```

Câștigi două lucruri deodată:

- **catch fin** unde te interesează diferența: `catch (NotFoundException)` prinde doar lipsa;
- **catch gros** unde nu te interesează: `catch (AcademyException)` prinde tot ce e „greșeală de-a noastră, prevăzută” — și **nu** prinde un `NullReferenceException`, care e un bug și trebuie să se vadă.

Azi `catch (ArgumentException)` nu-ți dă niciuna din cele două: e prea gros pentru prima și prea îngust pentru a doua.

Ordinea contează. Compilatorul te obligă la specific înainte de general:

```
catch (NotFoundException ex) { ... }
catch (AcademyException ex) { ... }
```

Invers, primul `catch` le-ar înghiți pe toate, iar al doilea ar fi cod mort — și primești eroare de compilare.

4. Excepția poate căra date, nu doar text

Un mesaj e un șir: ca să afli din el ce entitate lipsea, l-ai sparge cu `Split`. În schimb:

```
public class NotFoundException : AcademyException
{
    public string Entitate { get; }
    public string Cheie { get; }
}
```

Acum `catch` -ul poate lua decizii: `if (ex.Entitate == "Cartea")`. Textul rămâne pentru om, datele rămân pentru program.

5. `innerException` — excepția care nu-și pierde cauza

În tot proiectul tău nu apare nicăieri. E constructorul cu două argumente:

```
public FisierCoruptException(int linie, string continut, Exception cauza)
    : base("Linia " + linie + " nu poate fi citita", cauza)
```

Rostul: în `UserTextMapper.FromText`, un `Guid.Parse` pe un fișier stricat aruncă `FormatException` — un mesaj despre un șir, fără nicio urmă despre *care linie* din `users.txt`. Îl prinzi, arunci excepția ta cu numărul liniei, și **păstrezi originalul** dedesubt.

Fără `cauza`, informația se pierde definitiv. Cu ea, ai și contextul tău, și eroarea tehnică:

```
[date] Fisierul de date e stricat la linia 2.
cauza reala: FormatException - The input string 'o-mie-noua-sute' was not
in a correct format.
```

6. Când NU folosești excepții

`ViewLogIn.cs:50` aruncă pentru o parolă greșită. Dar o parolă greșită **nu e o situație excepțională** — e unul dintre cele două rezultate normale ale unui login. Un utilizator care greșește parola face exact ce se aștepta de la el.

Testul: *se poate întâmpla în funcționarea obișnuită a programului?* Dacă da, e un **rezultat**, și se întoarce. Dacă nu — fișierul de date corupt, un tip necunoscut într-un fișier scris de tine — e o excepție.

Diferența practică: un rezultat te obligă să-l tratezi acolo unde ești. O excepție sare peste tot ce e între `throw` și `catch` — și exact de-aia parola greșită iese azi din `Logger()`, trece prin `Main`, și ajunge pe ecran cu stack trace cu tot.

7. Proiectul de referință

`exemplu-exception/` din acest repo. Se compilează și rulează:

```
dotnet run
```

Are ierarhia completă, o excepție cu date proprii, `innerException` pe un fișier stricat, și login-ul făcut ca rezultat, nu ca excepție. Șapte scenarii, fiecare cu ce vede utilizatorul. **Uită-te în el, dar scrie tu codul din exerciții** — pe `academy`, nu pe el.

Exerciții ghidate

G1 — Ierarhia

Creează `Common/Exceptions/` cu `AcademyException` (abstractă, derivată din `Exception`) și trei clase derivate din ea: `ValidationException`, `NotFoundException`, `DuplicateException`.

Gata când: proiectul compilează și `catch (AcademyException)` prinde oricare din cele trei, verificat aruncând pe rând câte una.

G2 — Lipsa

Mută pe `NotFoundException` toate locurile care spun azi „nu exista in baza de date”:

`BookService.cs:46, :59` · `CourseService.cs:63, :77` · `UserService.cs:73, :87, :101, :117`
· `EnrolmentService.cs:61, :76`.

Atenție la `EnrolmentService.cs:61`: acolo scrie „nu **se afla** in baza de date”, nu „nu exista”. E aceeași situație, cu alt text — și e exact motivul pentru care nu poți căuta după mesaj. După tip, poți.

Gata când: niciunul dintre cele zece locuri nu mai aruncă `ArgumentException`, iar un `catch (NotFoundException)` le prinde pe toate zece.

G3 — Duplicatul

Același lucru pentru „se afla deja in baza de date”: `BookService.cs:33` · `CourseService.cs:50` · `UserService.cs:33, :45, :57` · `EnrolmentService.cs:48`.

Gata când: cele două grupuri sunt tipuri diferite, iar un `catch (NotFoundException)` nu mai prinde un duplicat.

G4 — Ce vede utilizatorul

În `ViewStudent.cs`, înlocuiește cele două `catch (ArgumentException ex)` cu `catch`-uri separate pe tipurile noi, fiecare cu mesajul lui. Scoate `Console.WriteLine(ex)`.

Gata când: adaugi o carte cu titlu gol și pe ecran apare o singură linie, fără nume de clasă, fără cale de fișier, fără număr de linie.

Practică

P1 — Excepția care cară date

Dă-i lui `NotFoundException` două proprietăți: ce entitate s-a căutat și după ce cheie. Folosește-le în `catch`.

Gata când: din `catch` poți scrie „Nu am gasit cursul: Algebra” fără să spargi mesajul cu `Split`.

P2 — Fișierul stricat

`Repository.Read()` cheamă `mapper.FromText(line)` pentru fiecare linie. Dacă o linie e stricată, azi aplicația moare la pornire cu o `FormatException` care nu spune care linie.

Scrie `FisierCorruptException` cu numărul liniei, conținutul ei și **cauza** originală (`innerException`). Aruncă-l din `Read()`.

Gata când: strici cu mâna linia 3 din `users.txt`, pornești, și mesajul spune „linia 3” și păstrează dedesubt `FormatException`-ul original.

P3 — Login-ul nu mai aruncă

Rescrie `ViewLogIn.Logger()` astfel încât utilizator inexistent și parolă greșită să fie **rezultate**, nu excepții.

Gata când: greșești parola de trei ori la rând și aplicația rămâne în ecranul de login, fără să treacă prin niciun `catch`.

Provocări

Doar cerința și condiția de gata. Fără pași.

C1 — O singură plasă

`Program.Main` are un singur `catch`. Fă-l să deosebească între o excepție de-a ta și un bug neprevăzut, și să nu arate niciodată utilizatorului un stack trace.

Gata când: pentru o excepție de-a ta se afișează o singură linie prietenoasă; pentru un `NullReferenceException` provocat intenționat se afișează un mesaj generic, iar detaliul tehnic ajunge în altă parte, nu în fața utilizatorului. Și: codul de ieșire al procesului e diferit în cele două cazuri.

C2 — Încărcarea nu mai cade pe o linie

O linie stricată în `books.txt` nu mai trebuie să împiedice pornirea aplicației. Celelalte cărți se încarcă, iar cele stricate sunt raportate la final.

Gata când: `books.txt` cu 7 rânduri, dintre care al 3-lea și al 5-lea stricate, pornește aplicația cu 5 cărți încărcate și un raport care numește liniile 3 și 5. Și: după o astfel de pornire, `git diff Data/` e gol — cele două linii stricate nu s-au pierdut de pe disc.

